



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo

INGENIERÍA EN SISTEMAS COMPUTACIONALES



UNIDAD DE APRENDIZAJE

Internet de las Cosas / Embedded Systems

PROFESOR

Aleman Arce Miguel Angel

Proyecto Final

Integrantes:

De la Rosa Gamiño Diego Emilio

López Ramos Marcos Isaid

Moran Vaquero Marcos

Grupo: 6CM3

Fecha de entrega: 20 de junio de 2026

1. Introducción

El presente reporte documenta el diseño, implementación y validación de un sistema de estacionamiento inteligente basado en una arquitectura IoT. El sistema utiliza una ESP32 como nodo embebido principal, un sensor ultrasónico HC-SR04 para detectar la presencia de un vehículo, el protocolo MQTT para comunicar los datos, Mosquitto como broker local y Node-RED Dashboard como plataforma de visualización.

La solución permite identificar si un cajón de estacionamiento se encuentra libre u ocupado mediante una medición de distancia. Cuando el objeto detectado se ubica a menos de 10 cm del sensor, el sistema considera el cajón como ocupado; cuando la distancia es mayor o no existe lectura válida, el sistema lo considera libre. La ESP32 también controla actuadores físicos para representar el estado: LED verde, LED rojo, buzzer y servo SG90.

2. Objetivo general

Diseñar e implementar una arquitectura IoT basada en MQTT que permita monitorear en tiempo real el estado de un cajón de estacionamiento y accionar dispositivos físicos de acuerdo con la información obtenida por un sensor ultrasónico.

3. Objetivos específicos

- Implementar un nodo publicador MQTT utilizando una ESP32.
- Medir distancia en tiempo real mediante el sensor HC-SR04.
- Configurar Mosquitto como broker MQTT local.
- Crear un Dashboard de monitoreo en Node-RED.
- Definir tópicos MQTT jerárquicos para distancia y estado.
- Controlar actuadores físicos: LEDs, buzzer y servomotor.
- Documentar la estructura para que el proyecto pueda replicarse con el hardware indicado.

4. Alcance del proyecto

El alcance del proyecto corresponde a un prototipo funcional a escala. El sistema no busca controlar un estacionamiento real con múltiples cajones, sino demostrar la integración completa entre hardware, firmware, red local, broker MQTT y dashboard de visualización. El modelo puede escalarse posteriormente agregando más nodos ESP32, más sensores o una base de datos para almacenar históricos.

5. Arquitectura general

La arquitectura sigue el modelo Publisher-Broker-Subscriber. La ESP32 actúa como publicador porque genera los mensajes de distancia y estado. Mosquitto funciona como intermediario de comunicación, y Node-RED se comporta como suscriptor para mostrar la información en una interfaz web.



Figura 1. Arquitectura general del sistema IoT.

6. Topología de red

La ESP32 se conecta por WiFi al mismo router o red local donde se encuentra la computadora que ejecuta Mosquitto y Node-RED. Esta decisión simplifica la comunicación durante la prueba de laboratorio, ya que no requiere servicios externos ni brokers en la nube.

Elemento	Función en la red
ESP32	Nodo IoT que mide distancia, controla actuadores y publica datos MQTT.
Router / red local	Medio de comunicación WiFi entre ESP32 y computadora.
Laptop / PC	Ejecuta Mosquitto como broker y Node-RED como dashboard.
Node-RED Dashboard	Interfaz web para visualizar estado, distancia e historial.

7. Componentes utilizados

Componente	Cantidad	Función
ESP32 Dev Module	1	Nodo principal IoT, publicador MQTT y controlador de actuadores.
HC-SR04	1	Sensor ultrasónico para medir la distancia del vehículo u objeto.
Servo SG90	1	Simulación de barrera automática.
LED verde	1	Indicador visual de cajón libre.
LED rojo	1	Indicador visual de cajón ocupado.
Buzzer activo	1	Alerta sonora proporcional a la cercanía del objeto.
Resistencias 220 ohm	2 o mas	Protección para LEDs y apoyo en el acondicionamiento de señales.
Protoboard y jumpers	Varios	Montaje y conexión del circuito.

8. Software utilizado

Software	Uso
Arduino IDE	Programación y carga del firmware en la ESP32.
ESP32 Arduino Core	Soporte de compilación para placas ESP32.
PubSubClient	Cliente MQTT para publicar mensajes desde la ESP32.
ESP32Servo	Control del servomotor SG90 desde la ESP32.
Mosquitto	Broker MQTT local para recibir y distribuir mensajes.
Node-RED	Construcción del flujo de suscripción y visualización.
Node-RED Dashboard	Interfaz gráfica web para mostrar estado, distancia e historial.

9. Conexiones del circuito

Las conexiones se definieron de forma directa para que el cableado sea fácil de replicar. El pin ECHO del HC-SR04 requiere atención especial, ya que el sensor puede entregar 5 V y la ESP32 trabaja con niveles lógicos de 3.3 V; por ello se recomienda usar divisor de voltaje para proteger GPIO18.

Elemento	Pin / terminal	Conexión ESP32
HC-SR04	VCC	VIN / 5V
HC-SR04	GND	GND
HC-SR04	TRIG	GPIO 5
HC-SR04	ECHO	GPIO 18 mediante divisor de voltaje
LED verde	Anodo	GPIO 26 con resistencia de 220 ohm
LED verde	Catodo	GND
LED rojo	Anodo	GPIO 27 con resistencia de 220 ohm
LED rojo	Catodo	GND
Buzzer	Positivo	GPIO 25
Buzzer	Negativo	GND
Servo SG90	Señal	GPIO 14
Servo SG90	VCC / GND	5V y GND común con ESP32

10. Lógica de funcionamiento

El firmware ejecuta un ciclo continuo: verifica la conexión MQTT, mide distancia, evalúa el criterio de decisión, actualiza actuadores y publica datos.

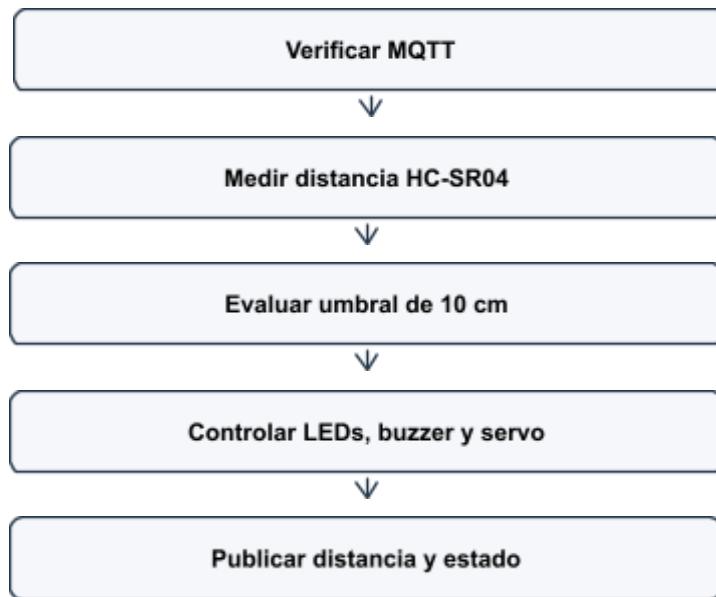


Figura 2. Flujo lógico del firmware en la ESP32.

Condición	Estado	LED verde	LED rojo	Buzzer	Servo
Distancia > 0 y < 10 cm	OCUPADO	Apagado	Encendido	Intermitente	0 grados
Distancia >= 10 cm o lectura no válida	LIBRE	Encendido	Apagado	Apagado	90 grados

11. Análisis del firmware ESP32

El archivo fuente del firmware es estacionamiento_esp32.ino. El programa está dividido en funciones con responsabilidades claras: medir distancia, conectar WiFi, reconectar MQTT y ejecutar el ciclo principal del sistema. Esta separación facilita que otra persona revise o modifique el proyecto.

La medición de distancia se realiza enviando un pulso de 10 microsegundos por TRIG y midiendo el tiempo de retorno en ECHO con pulseIn. La conversión utiliza la velocidad aproximada del sonido en aire, 0.0343 cm/us, y divide entre dos porque el pulso recorre ida y vuelta.

```

float medirDistancia() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duracion = pulseIn(ECHO_PIN, HIGH,

```

```

30000); if (duracion == 0) return -1;

return duracion * 0.0343 / 2;
}La conexión MQTT utiliza PubSubClient. Si la ESP32 se desconecta del broker, el
firmware intenta reconectarse cada tres segundos. Esta decisión mejora la continuidad del
sistema ante fallos temporales de red o reinicios del broker.

void reconectarMQTT() {
  while (!client.connected()) {
    Serial.print("Conectando MQTT... ");

    if
      (client.connect("ESP32_Estacionamient
        o")) { Serial.println("conectado");
      } else {
        Serial.print("Error rc=");
        Serial.println(client.state());
        delay(3000);
      }
    }
  }
}

```

12. Tópicos MQTT

Los tópicos se definieron de forma jerárquica para identificar institución, materia, equipo, sistema y variable publicada. Esta estructura facilita agregar más dispositivos o cajones en futuras versiones.

Tópico	Dato publicado	Ejemplo
escom/iot/equipo1/estacionamiento/distancia	Distancia medida por el sensor en centímetros.	16.25
escom/iot/equipo1/estacionamiento/estado	Estado calculado del cajón.	LIBRE u OCUPADO

En la implementación básica, PubSubClient publica mensajes con comportamiento equivalente a QoS 0. Para un prototipo de monitoreo en tiempo real esto es aceptable porque la distancia se actualiza constantemente; si un dato se pierde, el siguiente valor llega en pocos segundos.

13. Dashboard en Node-RED

El archivo flow_estacionamiento.json contiene el flujo de Node-RED. Este flujo se suscribe a los dos tópicos MQTT del proyecto y muestra la información en widgets de texto, medidor y gráfica histórica.

Tipo de nodo	Cantidad detectada
global-config	1
mqtt in	2
mqtt-broker	2
tab	1
template	1

Tipo de nodo	Cantidad detectada
ui_chart	1
ui_gauge	1
ui_group	2
ui_tab	2
ui_text	2

Elemento de Dashboard	Funcion
Texto de estado	Muestra LIBRE u OCUPADO.
Gauge de distancia	Representa la distancia actual en centímetros, con rango de 0 a 50 cm.
Grafica de historial	Muestra cambios de distancia en el tiempo.
Texto de distancia	Muestra la distancia numérica recibida por MQTT.

14. Justificación de decisiones de diseño

Decisión	Justificación técnica
Uso de MQTT	Es un protocolo ligero, adecuado para IoT, y separa al publicador de los suscriptores mediante un broker.
Mosquitto local	Permite realizar pruebas sin depender de internet ni de servicios externos.
Node-RED Dashboard	Facilita visualizar datos sin programar una interfaz desde cero.
Umbral de 10 cm	Es apropiado para un prototipo a escala, donde la presencia de un objeto cercano representa ocupación.
Buzzer proporcional	Aporta retroalimentación física adicional y permite notar mayor cercanía sin mirar el dashboard.

Servo SG90	Permite representar una barrera automática de forma simple y visible.
Separacion de tópicos	Facilita procesar distancia y estado por separado en Node-RED u otros clientes MQTT.

15. Procedimiento de ejecución

- Instalar Arduino IDE y el soporte ESP32 by Espressif Systems.
- Instalar las librerías PubSubClient y ESP32Servo desde el gestor de librerías.
- Instalar Mosquitto y configurarlo con listener 1883 y allow_anonymous true para pruebas locales.
- Instalar Node.js, Node-RED y node-red-dashboard.
- Importar flow_estacionamiento.json en Node-RED.
- Editar en estacionamiento_esp32.ino las variables ssid, password y mqtt_server.
- Cargar el firmware a la ESP32.
- Abrir el monitor serial a 115200 baudios.
- Abrir el Dashboard de Node-RED en <http://localhost:1880/ui>.

16. Pruebas realizadas

La validación del sistema se plantea mediante pruebas de conectividad, medición, actuadores y visualización. Al ejecutar el firmware, el monitor serial debe confirmar la conexión WiFi, la conexión MQTT y las lecturas de distancia con su estado correspondiente.

Prueba	Resultado esperado
Conexion WiFi	El monitor serial muestra WiFi conectado y la IP asignada a la ESP32.
Conexion MQTT	El monitor serial muestra Conectando MQTT... conectado.
Cajón libre	LED verde encendido, LED rojo apagado, buzzer apagado, servo a 90 grados y estado LIBRE.
Cajón ocupado	LED rojo encendido, LED verde apagado, buzzer intermitente, servo a 0 grados y estado OCUPADO.
MQTT por consola	mosquitto_sub recibe mensajes en los topicos de distancia y estado.
Dashboard Node-RED	Texto, gauge y gráfica se actualizan al cambiar la distancia.

17. Resultados

El sistema logra integrar correctamente los elementos principales de una arquitectura IoT: adquisición de datos físicos, procesamiento local, publicación de información, broker de comunicación y visualización en dashboard. La ESP32 publica la distancia medida por el HC-SR04 y el estado del cajón mediante MQTT; Node-RED recibe esos datos y los presenta en una interfaz web.

Los actuadores responden de acuerdo con el estado detectado. En estado libre, el sistema mantiene encendido el LED verde y abre la barrera. En estado ocupado, enciende el LED rojo, cierra la barrera y activa el buzzer con una frecuencia dependiente de la distancia.

18. Conclusiones

El proyecto demuestra que MQTT es una alternativa adecuada para sistemas IoT de monitoreo en tiempo real debido a su bajo consumo, simplicidad y facilidad para conectar productores y consumidores de datos. La combinación de ESP32, Mosquitto y Node-RED permite construir un prototipo completo sin infraestructura compleja.

Desde el punto de vista técnico, el sistema cumple con la detección de ocupación, comunicación por red local, visualización de datos y control de actuadores. Para una versión futura se recomienda agregar autenticación MQTT, almacenamiento histórico, una fuente externa estable para el servo y soporte para múltiples cajones de estacionamiento.

19. Anexos

Archivo	Descripción
README.md	Documentación general del repositorio y pasos para replicar el proyecto.
firmware/estacionamiento_esp32.ino	Firmware cargado en la ESP32.
node-red/flow_estacionamiento.json	Flujo importable para Node-RED Dashboard.

Comandos útiles para verificar MQTT desde la computadora donde se ejecuta Mosquitto:

- `mosquitto_sub -h localhost -t`
- `"escom/iot/equipo1/estacionamiento/estado" mosquitto_sub -h`
- `localhost -t "escom/iot/equipo1/estacionamiento/distancia"`
- `mosquitto_sub -h localhost -t "escom/iot/equipo1/estacionamiento/#"`